

Advanced Terraform on Azure

Lab6b Pipeline Automation

Expected Duration	2
Prerequisites.....	2
Part 0. Initiate DevOps Server VM Deployment	3
Part 1. Local PS Pipeline-Enforced Changes	3
Teaching Points	3
Phase 0 – Review the Promotion Model.....	4
Phase 1 – Review the Local Pipeline.....	4
Phase 2 - Pipeline-Applied Changes 1	5
Phase 3 - Pipeline-Applied Changes 2.....	5
Phase 4 – Verify Environment Order Enforcement	5
Phase 5 – Promote Through the Pipeline	6
Part 2. Automated Promotion with Azure DevOps (ADO) Pipelines	6
Teaching Points	6
Phase 1 – Complete the DevOps Server deployment	7
Phase 2 – Create the ADO Project Environment Roots Repository	13
Phase 3 – Configure Azure Authentication for Pipelines	15
Phase 4 – Configure ADO Environments and Approvals	17
Phase 5 – Add a Change Pipeline (Apply with approvals).....	17
Phase 6 – Release Network v1.0.2 to DEV	18
Phase 7 – Attempt an out-of-order promotion (expected to fail)	19
Part 3. Environment-Scoped Promotion Pipelines.....	21
Teaching Points	21
Phase 1 - Rethink Promotion Responsibility	22
Phase 2 - Add environment-scoped PR plan pipelines (dev/test/prod)	22
Phase 3 - Retire the change-all Pipeline	28
Phase 4 - Review and release Environment-Scoped Pipelines.....	28
End-to-End Perspective (Read Before Moving On)	30

Reflection: Something Still Is not Explicit	31
Part 4. Forensic Review of a Production Promotion	31
Teaching Points	31
Phase 1 – Create a New Module Version	32
Phase 2 – Promote new security module to PROD	32
Phase 3 – Pull Request and Validation	33
Phase 4 – Observe the Production Pipeline	34
Phase 5 – Approval and Execution	34
Phase 6 – Forensic Review	35
Why This Matters	37
Transition to next lab.....	38

Expected Duration

Part 1 of this lab is expected to take between 10-15 minutes

Part 2 of this lab is expected to take 30-40 minutes

Part 3 of this lab is expected to take 40-50 minutes

Part 4 of this lab is expected to take 30-40 minutes

Total expected lab time is therefore between 110 - 145 minutes

Take breaks between parts as needed.

Prerequisites

This lab assumes the presence of a remote state backend, as created in Lab 4. If you have not created the remote state backend **today** then run the following script ..

```
C:\qatfadvez\artifacts\fixes\bootstrap-backend.ps1
```

This lab also assumes that Lab6a has been completed **today**. If this is not the case then run the following commands from a VSC terminal session ...

```
cd c:\qatfadvez\lab6\original\reset6b
.\reset6b.ps1
```

You will also need to re-clone a copy of the network and security module repos you created in Lab6a..

```
cd c:\
mkdir module-repos
cd module-repos
git clone {your network repo url}
git clone {your security repo url}
```

Re-add your repo folder to VSC Explorer

Part 0. Initiate DevOps Server VM Deployment

Restrictions in the lab environment prevent the use of traditional web-based DevOps. A work-around to this is the use of a stand-alone DevOps Server which provides all of the functionality needed for controlled pipelining.

The deployment of a DevOps Server virtual machine takes approximately 20 minutes to complete. Therefore, you will initiate the vm deployment and leave it to run in the background ahead of its configuration and use in Part 2 of this lab.

1. Review the deployment in **artifacts\ado** and then update **terraform.tfvars** with your subscription id.
2. Run ..

```
cd c:\qatfadavaz\artifacts\ado-vm
terraform init
terraform apply --auto-approve
```

3. Leave the deployment to run and continue with Part 1.

Part 1. Local PS Pipeline-Enforced Changes

Teaching Points

So far you have promoted module versions manually across DEV, TEST, and PROD. You made explicit decisions about when promotion should occur and represented promotion by changing pinned module versions.

Now we introduce **pipelines** — not to make decisions for you, but to **enforce the rules you already defined**.

Here we demonstrate how platform teams prevent mistakes by automating *execution*, while keeping *decision-making* firmly in human hands.

By the end of these steps , you will understand how to:

- Use pipelines to enforce Terraform promotion rules
- Separate promotion decisions from execution
- Prevent unsafe or out-of-sequence environment changes
- Replace manual Terraform execution with controlled automation
- Prepare for full CI/CD adoption without changing architecture

When people hear the word *pipeline*, they often think of GitHub Actions or Azure DevOps. That is **not** what this part of the lab introduces, that comes in Part 2. Here, a *pipeline* simply means:

A repeatable sequence of commands that a machine runs instead of a human, with rules enforced automatically.

This part of the lab is organised into the following phases:

Phase 0 – Review the Promotion Model

Phase 1 – Review the Local Pipeline

Phase 2 – Pipeline-Applied DEV Deployment

Phase 3 – Enforce Promotion Order

Phase 4 – Promote through the pipeline

Phase 0 – Review the Promotion Model

Before introducing automation, the following rules have been established

- Promotion is a module version pin change
- DEV validates new module versions
- TEST cannot advance ahead of DEV
- PROD cannot advance ahead of TEST

Promotion also includes reverting a module version pin

Phase 1 – Review the Local Pipeline

In this phase, you introduce a **local pipeline script**.

This script replaces you typing Terraform commands by hand.

The pipeline will:

- Run Terraform in a fixed order
- Enforce promotion rules
- Refuse to run if rules are violated
- Apply Terraform non-interactively

The pipeline does **not**:

- Decide when to promote
- Change module versions
- Auto-promote environments
- Modify Terraform structure

You have been given a set of PowerShell scripts at **artifacts/pipelines/powershell**. These scripts replace manual terraform init / plan / apply operations.

Phase 2 - Pipeline-Applied Changes 1

A decision has been made to promote network module version **v1.0.0** in all environments.

1. Update the pinned version of the **network** module to **v1.0.0** in **main.tf** of each of the environments
2. Open a new terminal session and run each of these in turn ...
 - c:\qatfadvaz\artifacts\pipelines\powershell\change-dev.ps1
 - c:\qatfadvaz\artifacts\pipelines\powershell\change-test.ps1
 - c:\qatfadvaz\artifacts\pipelines\powershell\change-prod.ps1
3. Observe:
 - Terraform runs without manual input
 - Commands execute in a fixed order
4. In the Portal, verify the VNet manager tag is 'Michael Coulling-Green (NetMod v1.0.0)'

Phase 3 - Pipeline-Applied Changes 2

A decision has been made to promote network module version **v1.0.1** in the DEV environment.

5. Update the pinned version of the **network** module to **v1.0.1** in **main.tf** of the **dev** environments
6. Run ...
 - c:\qatfadvaz\artifacts\pipelines\powershell\change-dev.ps1**
7. In the Portal, verify the VNet manager tag updates to "John Robinson (NetMod v1.0.1)" from 'Michael Coulling-Green (NetMod v1.0.0)' in the dev resource group.

Phase 4 – Verify Environment Order Enforcement

In this phase, you observe how the pipeline prevents unsafe promotion by trying to promote v1.0.1 of the network module to PROD, bypassing TEST.

8. Update the pinned version of the **network** module to **v1.0.1** in **main.tf** of the **prod** environments

9. Run ...

```
c:\qatfadvas\artifacts\pipelines\powershell\change-prod.ps1
```

10. Observe:

- The pipeline fails
- Terraform does not run
- A clear error message is displayed

```
OperationStopped: (Promotion block...align versions)::Strin  
Promotion blocked: prod (1.0.1) is ahead of test (1.0.0).
```

11. This demonstrates that the pipeline enforces rules — it does not assume intent.

Phase 5 – Promote Through the Pipeline

12. Promote network module v1.0.1 to TEST and then PROD using the pipeline.

Takeaways

- Pipelines enforce behaviour — they do not decide promotion
- Automation protects humans from mistakes
- Promotion remains intentional and auditable
- Terraform architecture did not change
- This model scales directly into CI/CD systems

In the next part of this lab, you will move this same promotion logic into a **real CI/CD system**, without changing the model you have just learned.

Part 2. Automated Promotion with Azure DevOps (ADO) Pipelines

Teaching Points

In Part 1 you performed promotion using a local pipeline. In this part of the lab, you take automating the **execution** of those same steps to using **Azure DevOps (ADO) Pipelines**, while keeping **promotion decisions and approvals visible and human-controlled**.

This lab remains **Pattern A**: parallel roots (DEV/TEST/PROD) with parallel state. You are **automating execution**, not changing environment strategy.

Network and security modules remain independent. In this lab you promote a new **network module version only**. The security module remains pinned.

The objective of this lab is to promote **network v1.0.2** through **DEV→TEST→PROD** using ADO Pipelines, with approvals.

This part of the lab is organised into the following phases:

Phase 1 – Complete the DevOps Server deployment

Phase 2 – Create the ADO Project and Environment Roots Repository

Phase 3 – Deploy Self Hosted ADO Pipeline Agent

Phase 4 – Create Azure Authentication for Pipelines

Phase 5 – Configure ADO Environments and Approvals

Phase 6 – Add a Promotion Pipeline (Apply with approvals)

Phase 7 – Release Network v1.0.2 to dev

Phase 8 – Attempt an out of order Promotion

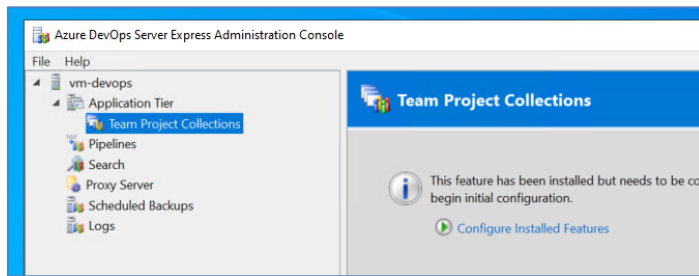
Phase 1 – Complete the DevOps Server deployment

This lab assumes **Lab 6a** was successfully completed.

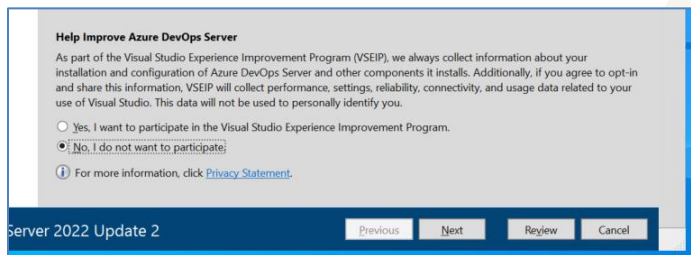
Confirm the following before you begin:

- You have three deployed environments: envs/dev, envs/test, envs/prod (Pattern A)
 - All environments currently consume **network v1.0.1** and **security v1.0.0**
 - Your remote backend already exists and holds:
 - dev.tfstate
 - test.tfstate
 - prod.tfstate
 - You have two module repositories:
 - terraform-azure-module-network
 - terraform-azure-module-security
1. Switch to the terminal session in which you deployed the DevOps Server VM. Once output is shown, switch to the Azure Portal and restart **vm-devops** in the resource group **rg-adoagent**
 2. Wait one minute for the **vm-devops** to restart
 3. Access **vm-devops** over RDP using the output IP address shown and **azureuser/TF_P@ssw0rd_123** Reduce the display configuration to a resolution lower than your vm display resolution to avoid 'full-screen' issues.
 4. Click **Yes** for Network discoverability.
 5. Dismiss Server Manager (Any errors shown are benign 'delayed-start' services)
 6. Navigate to and open Azure DevOps Server Express Administration Console

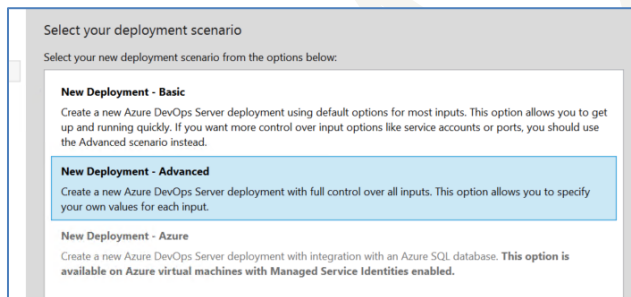
7. Navigate to Application Tier > Team Project Collections and select 'Configure Installed Features' ...



8. Click on 'Start Wizard'
9. Select '**No, I don't want to participate**' under Help Improve Azure DevOps Server ...

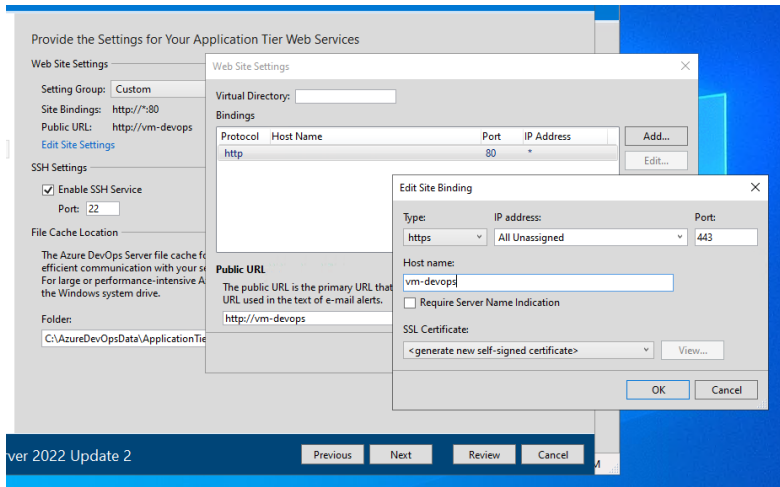


10. Click on **Next** twice and then select '**New Deployment - Advanced**' then **Next** ...

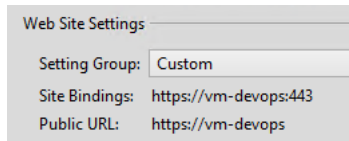


11. Click **Next** confirming English
12. Click **Next**, confirming **vm-devops\SqLExpress** as the database
13. Click **Next**, confirming use of Local Service system account
14. For Web Site Settings, Select '**Edit Site Settings**'
15. Change the Public URL to 'https://vm-devops'
16. Select the http bindings and click on **Edit**
17. Change the Type to '**https**'
18. Set the Host name as '**vm-devops**'
19. Uncheck 'Require Server Name Indication'

20. Click OK



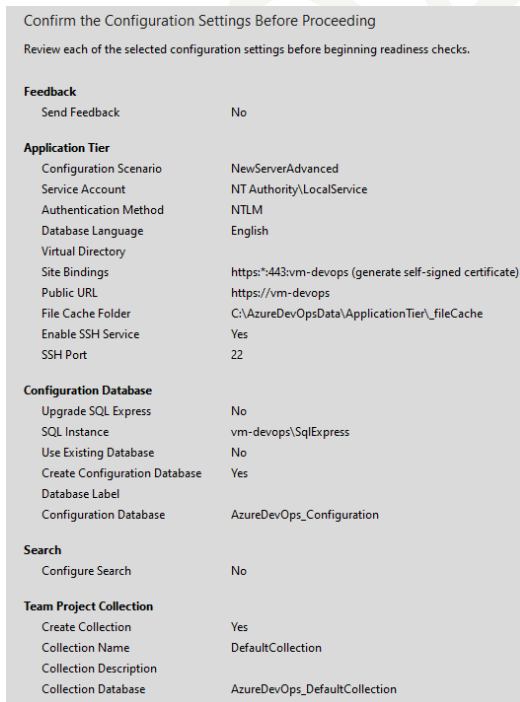
21. Verify the Web Site Settings before clicking **Next ...**



22. Uncheck 'Install and configure Search' then click Next

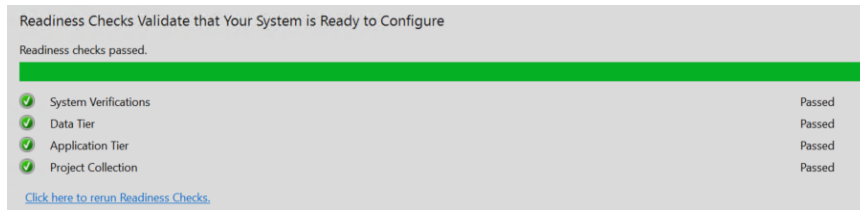
23. Click **Next**, confirming new collection name '**DefaultCollection**'

24. Review the settings ...



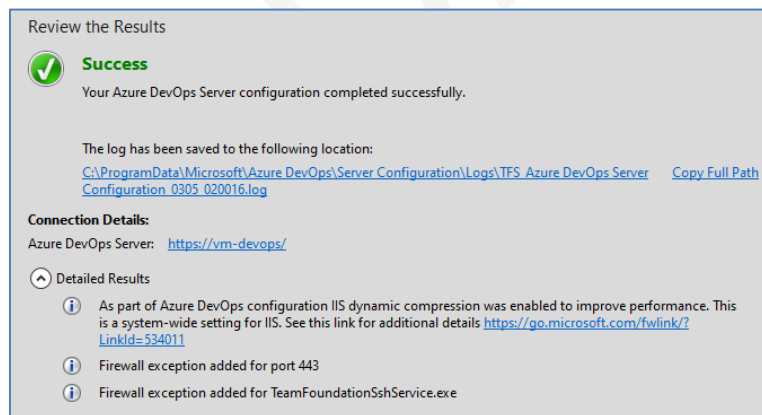
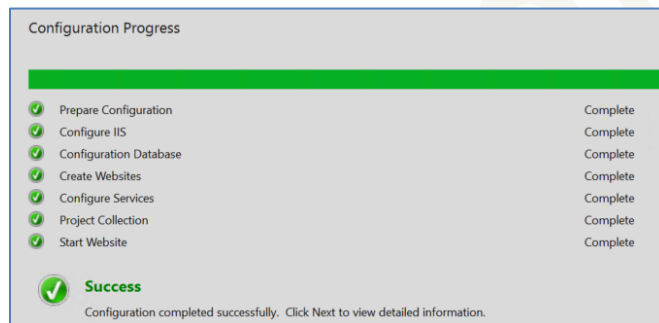
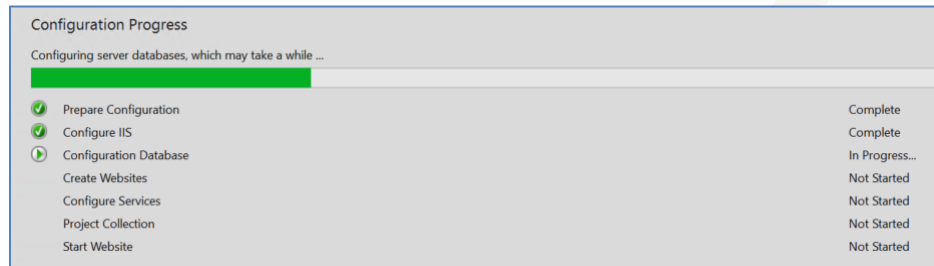
25. Click Next

26. Readiness checks should complete successfully ..



27. Click Configure

28. The Database configuration phase will take 6-10 minutes, grab a break ...



29. Once complete, close the Server Configuration Wizard

30. Close the ADO Administration Console

31. Switch to your student desktop vm ...

32. Run notepad as administrator and open **c:\windows\system32\drivers\etc\host**

33. Add an entry for vm-devops {vm public ip of **your** vm-devops}

```
# localhost name resolution is handled within DNS itself.  
#      127.0.0.1      localhost  
#      ::1            localhost  
20.90.248.109 vm-devops
```

34. Save the changes

35. Open a web browser session to **https://vm-devops**

36. You will receive a 'Your connection is not private' message

37. Click on 'Not secure' in the URL bar and select **Certificate details**

38. Select the Details tab and click on '**Export**'

39. Save vm-devops.crt to Downloads

40. Navigate to Downloads and double-click **vm-devops.crt**

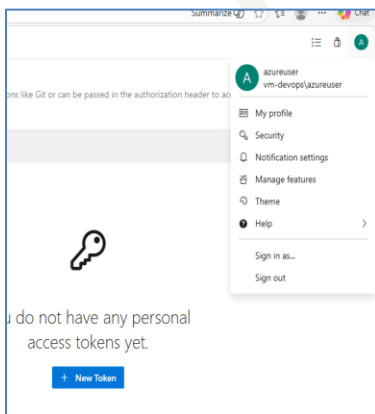
41. Select Install Certificate > Local Machine > Place all certificates > Trusted Root Certificate Authorities > Next > Finish

42. Bookmarking existing links as needed, Close your existing browser session, open a new one and browse to **https://vm-devops**

43. Log in using azureuser/TF_P@ssw0rd_123

Create a PAT

44. Click your account Avatar > Security > New Token ..



45. Name the token '**usertoken**', select 'All accessible organizations' and grant Full Access ..

Create a new personal access token

Name: azureusertoken

Organization: All accessible organizations

Expiration (UTC): 30 days, 4/4/2026

Scopes: Authorize the scope of access associated with this token. Scopes: ☒ Full access, ☐ Custom defined

46.

47. **Note:** In production environments, PAT scopes should be restricted according to the principle of least privilege.

48. Click Create

49. Copy the token and save it for later use

Install tools

50. Switch back to the vm-devops RDP session

51. Open an elevated powershell session and run ..

```
irm https://raw.githubusercontent.com/qatip/ado-bootstrap/main/stage1-install-tools.ps1 | iex
```

Note: Ensure “ado-bootstrap”, not “adobootstrap” is entered. Copy and paste may drop the “-“

For reference. scripts are at <https://github.com/qatip/ado-bootstrap.git>

52. VM will reboot

53. Wait one minute before reconnecting over RDP

54. Dismiss Server Manager if it opens

Install and configure the agent

55. Open an elevated powershell session

```
irm https://raw.githubusercontent.com/qatip/ado-bootstrap/main/stage2-install-agent.ps1 | iex
```

Note: Ensure “ado-bootstrap”, not “adobootstrap” is entered. Copy and paste may drop the “-“

56. run ./config.cmd

57. Configure as shown

URL: https://vm-devops/DefaultCollection

Agent Pool: {enter}

Agent Name: {enter}

Work folder: {enter}

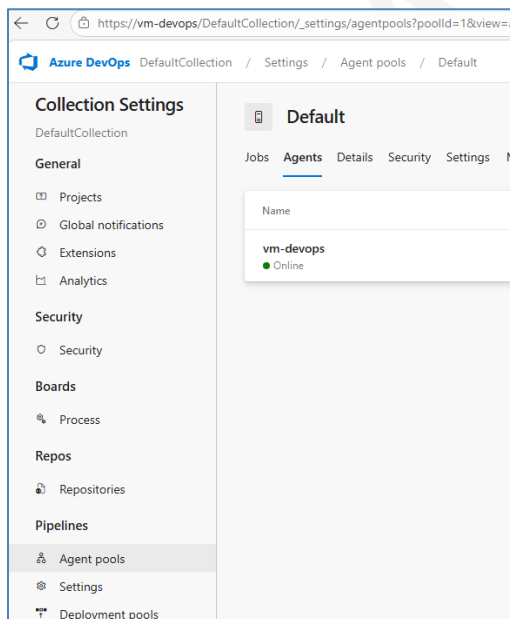
Run as Service: **Y**
Enable unrestricted: **Y**
User account: **{enter}**
Prevent service start: **N**

```
Administrator: Windows PowerShell

>> Connect:
Enter server URL > https://vm-devops/DefaultCollection
Enter authentication type (press enter for Integrated) > PAT
Enter personal access token > *****
Connecting to server ...

>> Register Agent:
Enter agent pool (press enter for default) >
Enter agent name (press enter for vm-devops) >
Scanning for tool capabilities.
Connecting to the server.
Successfully added the agent
Testing agent connection.
Enter work folder (press enter for _work) >
2026-03-22 01:17:15Z: Settings Saved.
Enter run agent as service? (Y/N) (press enter for N) > Y
Enter enable SERVICE_SID_TYPE_UNRESTRICTED for agent service (Y/N) (press enter for N) > Y
Enter User account to use for the service (press enter for NT AUTHORITY\NETWORK SERVICE) >
Granting file permissions to 'NT AUTHORITY\NETWORK SERVICE':
Service vstsagent.vm-devops.Default.vm-devops successfully installed
Service vstsagent.vm-devops.Default.vm-devops successfully set recovery option
Service vstsagent.vm-devops.Default.vm-devops successfully set to delayed auto start
Service vstsagent.vm-devops.Default.vm-devops successfully set SID type
Service vstsagent.vm-devops.Default.vm-devops successfully configured
Enter whether to prevent service starting immediately after configuration is finished? (Y/N)
Service vstsagent.vm-devops.Default.vm-devops started successfully
PS C:\ado-agent>
```

58. You can now sign out of the RDP session
59. In your student-vm browser session, verify the agent is registered and online



Phase 2 – Create the ADO Project Environment Roots Repository

60. Click on 'Azure DevOps' to return to the DefaultCollection landing page

61. Create a new project named '**terraform-lab**'
62. Inside the project, create a new Git repository: **env-roots**
63. Delete the default 'terraform-lab' rep
64. Open Credentials Manager in Control Panel and 'Add a Windows credential' as shown, using password TF_P@ssw0rd_123 ...

Type the address of the website or network location and your credentials

Make sure that the user name and password that you type can be used to access the location.

Internet or network address (e.g. myserver, server.company.com):	vm-devops
User name:	azureuser
Password:	TF_P@ssw0rd_123

65. Clone the new empty repository locally ...

```
git config --global http.sslBackend schannel
git config --global http.sslVerify true
cd\
mkdir c:\ado-repo
cd c:\ado-repo
git clone https://vm-devops/DefaultCollection/terraform-
lab/_git/env-roots
cd env-roots
```

66. Add the repo folder to VSC Explorer; File > Add Folder to Workspace > c:\ado-repo
67. To ensure correct credentials are used against the repo, run ..
68. git config --local credential.useHttpPath true
69. git config --local user.name "azureuser"
70. git config --local user.email "azureuser@company.com"
71. Copy your **lab6/guided/envs** folder into this repository.

Add a .gitignore

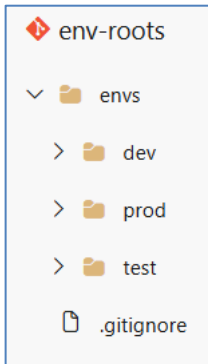
72. Create a .gitignore with:

```
.terraform/
*.tfstate
*.tfstate.*
crash.log
```

In this course, .tfvars files are committed intentionally.
They define environment configuration and are reviewed as part of promotion.

73. Commit and push:

```
cd c:\ado-repo\env-roots
git add .
git commit -m "Add environment roots (dev/test/prod)"
git push -u origin main
```



Phase 3 – Configure Azure Authentication for Pipelines

In this phase you create a **Service Connection** in DevOps, used so that pipelines can run Terraform, has been pre-created for you in the lab environment.

74. Create create a service principal, update with your subscription and a 4 random digit unique identifier

```
az ad sp create-for-rbac --name "ado-###" --role Contributor --
scopes /subscriptions/{subscription-id}
```

```
{
  "appId": "ee7c5602-d557-468d-bf00-c0605b943cbc",
  "displayName": "ado-1234",
  "password": "Y_f8Q~WtK_m17My_MnUzzUTNRepG5yYMLyjcZakh",
  "tenant": "4cfe372a-37a4-44f8-91b2-5faf34253c62"
}
```

New service connection

Choose a service or connection type

Search connection types

- ☐ Azure Classic
- ☐ Azure Repos/Team Foundation Server
- ☒ Azure Resource Manager

New Azure service connection ×

Azure Resource Manager

Authentication method

- ☒ Service principal (manual)

[Need help choosing a connection type?](#) Back Next

Showing subscriptions in LODS-Prod-MCA directory. Don't see a subscription? [Switch directory](#)

Search for any file... Subscriptions : **Filtered (1 of 1)** My role == all

Subscription name ↑↓	Subscription ID ↑↓
QA--lod52281922	7da9ccfa-28ad-4177-9dde-ff6e259f6702

75. Navigate to Project Settings > Service connections > Create service connection

- Choose Azure Resource Manager
- Use automatic App registration
- Authenticate when requested
- Scope to your subscription
- Name: sc-terraform-lab
- Authorise for all pipelines

Backend configuration

76. Retrieve your current Terraform remote backend values:

- storage_account_name
- container_name
- resource_group_name

77. Create a variable group (Pipelines > Library) ...

78. Name: vg-terraform-backend

79. Variables:

```
TF_BACKEND_SA : advtfstate{your suffix}
TF_BACKEND_CONTAINER : tfstate
TF_BACKEND_RG : landing-zone-shared-tfstate-rg
ARM_SUBSCRIPTION_ID: {your subscription id}
```

80. Save changes

Phase 4 – Configure ADO Environments and Approvals

This phase enables **controlled promotion** using approvals.

81. Create three environments (Pipelines > Environments) :

- dev
- test
- prod

82. Add approvals (refresh screen if permission error initially displayed):

- Leave **dev** without approval
- Add an approval (yourself) to **test**
- Add an approval (yourself) to **prod**

83. This mirrors real workflows: fast feedback in dev, deliberate control later.

Phase 5 – Add a Change Pipeline (Apply with approvals)

84. A pipeline that applies Terraform in DEV, then TEST, then PROD, enforcing execution order and approvals where needed has been provided in the course-wide **artifacts**.

85. Create folder **.azure-pipelines** under env-roots

86. Copy the supplied **change-all.yml** file:

- From: artifacts/pipelines/change
- To: env-roots/.azure-pipelines

87. From **ado-repo\env-roots**, commit and push to main ...

```
cd c:\ado-repo\env-roots
git add .azure-pipelines/change-all.yml
git commit -m "Add change-all pipeline (apply with approvals)"
```

`git push`

88. In DevOps > Pipelines > Create Pipeline ...

- Azure Repos Git
- Repository: **env-roots**
- Existing YAML file
- Branch: **main**
- Path: .azure-pipelines/change-all.yml

89. **Save** the pipeline without running it. (drop-down to right of Run)

90. Rename the pipeline from '**env-roots**' to '**change-all**'

Phase 6 – Release Network v1.0.2 to DEV

Release v1.0.2 of the network module

91. In module-repos/terraform-azure-module-network, make a small non-breaking change ...

In local.tf, update the manager tag from 'John Robinson (NetMod v1.0.1)' to '**Clare Hooper (NetMod v1.0.2)**'

92. At terraform-azure-module-network, Commit, Tag and Push ...

```
cd c:\module-repos\terraform-azure-module-network
git add locals.tf
git commit -m "Publish network module v1.0.2"
git tag v1.0.2
git push origin main
git push origin v1.0.2
```

Promote Network v1.0.2 to DEV

93. In **ado-repo/env-roots/envs/dev**, update **main.tf** ...

Change the **network** module source: ref=v1.0.1 → **ref=v1.0.2**

94. At **env-roots**, commit and push to main ...

```
cd c:\ado-repo\env-roots
git add .\envs\dev\main.tf
git commit -m "Promote Network Module v1.0.2 to DEV"
git push
```

95. The change-all pipeline is triggered on updates to the main branch of the repo and therefore should now be in progress. Navigate to Pipelines>Recent and click into change-all...

- Grant the permissions request (one-off for initial run of a pipeline)
- Dev applies automatically without approval
- Test waits for permissions and approval - Grant and Approve
- Prod waits for permissions and approval - Grant and Approve

96. Observe the runs and verify the changes to DEV and no-ops to TEST and PROD...

← Jobs in run #20260107.1		
promote-all		
Apply - dev		
>	✓ Terraform apply (dev)	1m 33s
Apply - test (approval)		
>	✓ Terraform apply (test)	1m 31s
Apply - prod (approval)		
>	✓ Terraform apply (prod)	1m 21s

Drill into the TEST run to verify no gate block, TEST is not ahead of DEV

```
Detected module versions (from main.tf):
  network dev=1.0.2 test=1.0.1
  security dev=1.0.0 test=1.0.0
Gate passed: test is not ahead of dev for any governed module.
Finishing: Gate: block test if ahead of dev (all governed modules)
```

Drill into the PROD run to verify no gate block, PROD is not ahead of TEST

```
Detected module versions (from main.tf):
  network test=1.0.1 prod=1.0.1
  security test=1.0.0 prod=1.0.0
Gate passed: prod is not ahead of test for any governed module.
Finishing: Gate: block prod if ahead of test (all governed modules)
```

Phase 7 – Attempt an out-of-order promotion (expected to fail)

Attempt to promote v1.0.2 to Prod

97. In env-roots, update **envs/prod/main.tf** ...

Change the network source: ref=v1.0.1 → **ref=v1.0.2**

98. At **env-roots**, commit and push to main ...

```
cd c:\ado-repo\env-roots
git add .\envs\prod\main.tf
```

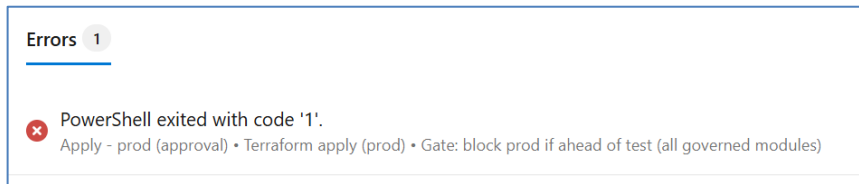
```
git commit -m "Promote Network Module v1.0.2 to PROD"
git push
```

99. The pipeline triggers:

Dev applies automatically (no-op)

Approve Test (no-op)

Approve Prod (**failure expected**)



Drill into the PROD run to verify gate block, PROD is ahead of TEST

```
Detected module versions (from main.tf):
network test=1.0.1 prod=1.0.2
Promotion blocked: prod network (1.0.2) is ahead of test network (1.0.1). Promote in order.
```

100. Revert the incorrect promotion commit to main (restore prod to network v1.0.1)

```
cd c:\ado-repo\env-roots
git revert --no-edit HEAD
git push
```

101. This will trigger a no-op pipeline run. For expedience, cancel the run. (the run, do not delete the pipeline!)

102. In **env-roots**, update the environment configuration to reflect the correct promotion order:

Update the network module to v1.0.2 in envs/test/main.tf and envs/prod/main.tf

103. Commit and push to main ...

```
cd c:\ado-repo\env-roots
git add .\envs\test\main.tf
git add .\envs\prod\main.tf
git commit -m "Promote Network Module v1.0.2 to TEST/PROD"
git push
```

104. Observe the promotion pipeline ...

- Dev runs automatically (expected no-op)
- Approve Test
- Approve Prod

105. **Confirm:** Network module v1.0.2 has been promoted through DEV/TEST/PROD
106. **Confirm:** Security module v1.0.0 is still current through DEV/TEST/PROD

Key Takeaways

- ADO Pipelines automate **execution**, not decision-making
- Approvals keep promotion **intentional and auditable**
- Pattern A remains unchanged
- Modules are promoted independently
- Deterministic no-op applies build trust in automation

Where This Design Stops

We have intentionally automated **execution across environments** while leaving release intent **implicit**. Because a single central promotion pipeline is used:

- All environments run, even if only one changed
- Invalid promotion intent can exist briefly in Git
- Humans must correct mistakes (for example, reverting a pin)

These limitations are **intentional** at this stage of the course.

Part 3. Environment-Scoped Promotion Pipelines

Teaching Points

In Part 2 you automated Terraform execution across DEV, TEST, and PROD using a single promotion pipeline with approval gates. That design deliberately centralised execution so that ordering, approvals, and Terraform behaviour could be observed clearly.

However, it also made release intent implicit: the pipeline decided *where* to run, rather than *why* a promotion should occur.

That demonstrated *how* promotion can be executed safely and audibly, but it also deliberately exposed an important limitation: **automation alone does not enforce release intent**.

In this part, you evolve the design to a **production-grade promotion model** by introducing **one pipeline per environment**. Each environment advances **only when explicitly promoted**, preventing accidental unintentional leapfrogging and aligning automation with real-world release practices.

This part teaches the distinction between:

- execution automation
- release intent
- environment ownership

This part of the lab is organised into the following phases:

Phase 1 - Rethink Promotion Responsibility

Phase 2 - Add environment-scoped PR plan pipelines (dev/test/prod)

Phase 3 - Retire the change-all Pipeline

Phase 4 - Review and release Environment-Scoped Pipelines

Phase 1 - Rethink Promotion Responsibility

Before changing any pipelines, pause and reflect on the following principle:

Automation should never decide when to promote — only how to execute a promotion once the decision is made.

Previously:

- a single pipeline executed Terraform for all environments
- approvals gated execution
- but **release intent was implicit**

Moving forward:

- promotion intent becomes explicit
- each environment advances independently
- time separation between dev, test, and prod is restored

Phase 2 - Add environment-scoped PR plan pipelines (dev/test/prod)

A *plan pipeline* runs terraform plan automatically when a change is proposed, allowing humans to review what *would* change before anything is applied. It does not deploy resources or make decisions — it exists purely to make change visible early. These pipelines run terraform init and terraform plan against the target environment's state but never apply changes.

Earlier, plan pipelines were deliberately omitted because the focus was on understanding *execution behaviour*: how Terraform applies changes, how ordering and approvals work, and how environments converge safely. At that stage, adding PR-time validation would have distracted from the core lesson.

Now that environments are to advance independently, visibility must also become environment-specific. A change to DEV, TEST, or PROD represents a different level of risk and intent, and each requires its own review. Environment-scoped plan pipelines provide that visibility without changing how promotion itself works.

In this course we want the plan to be reviewed **before intent is declared**. We will therefore run Terraform plans automatically during the pull request (as an *optional* PR check) so reviewers can pause and consider impact while the change is still only a proposal or skip the planning if desired.

Merging the PR is the **explicit decision to apply the changes**. After merge, environment-scoped change pipelines run, and approvals act as the final control point before apply.

Objective

We want pull request validation to be environment-specific:

- Changes under envs/dev/** trigger a **dev** Terraform plan
- Changes under envs/test/** trigger a **test** Terraform plan
- Changes under envs/prod/** trigger a **prod** Terraform plan

The operator reviews the plan output and decides whether to merge.

Why environment-scoped PR plan pipelines?

Using one PR plan pipeline per environment keeps validation deterministic and easy to reason about.

A pull request touching dev should not run test or prod plans. There is no environment detection logic and no branching behaviour inside a pipeline.

This avoids brittle “detect which environment changed” scripting and keeps the model auditable, scalable, and aligned with the environment-scoped promotion pipelines introduced later in this lab.

Add the three env PR plan YAML files into the env-roots repo

In your local clone of **env-roots**:

1. Create a branch:

```
cd c:\ado-repo\env-roots
git checkout main
```

```
git pull
git branch -D add-pr-plans
git checkout -b add-pr-plans
```

2. Copy these files from artifacts/pipelines/plan into env-roots/.azure-pipelines/ ...

```
pr-plan-dev.yml
pr-plan-test.yml
pr-plan-prod.yml
```

3. Commit + push:

```
cd c:\ado-repo\env-roots
git add .azure-pipelines/pr-plan-dev.yml
git add .azure-pipelines/pr-plan-test.yml
git add .azure-pipelines/pr-plan-prod.yml
git commit -m "Add PR Plan pipelines"
git push -u origin add-pr-plans
```

4. In DevOps > Repos > env-roots > Pull requests > **Create a pull request** to main and **Complete (merge)** it.

Checkpoint: In Repos > env-roots > main > Files, you can see all three YAML files under .azure-pipelines/...

For each new YAML file, create a pipeline reference in Azure DevOps.

5. DevOps > Pipelines > New pipeline

6. Choose Azure Repos Git

7. Select repo: **env-roots**

8. Choose Existing Azure Pipelines YAML file

9. Branch: main

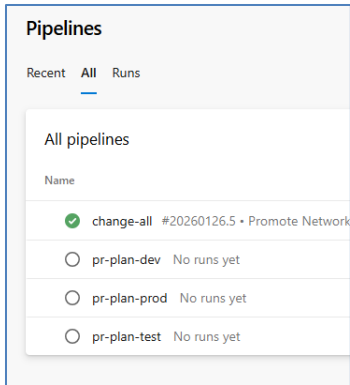
10. Path: select each of these in turn:

- .azure-pipelines/pr-plan-dev.yml
- .azure-pipelines/pr-plan-prod.yml
- .azure-pipelines/pr-plan-test.yml

11. Rename the pipeline from env-roots to one of these as appropriate:

- pr-plan-dev
- pr-plan-prod
- pr-plan-test

12. **Checkpoint:** You now have three new named pipelines listed under Pipelines...



Configure PR plan pipelines as optional checks

To ensure PR plan pipelines run automatically and are visible to reviewers, we attach them to the main branch as **optional build validation policies**.

These policies:

- make plan output visible during PR review
- do **not** block merges
- do **not** enforce governance yet

This preserves flexibility while making intent review possible.

13. DevOps > Repos > Branches

14. Locate branch: **main**

15. Select ⋮ More options → Branch policies

16. Under Build validation, click Add

Configure:

Build pipeline: pr-plan-dev

Path filter (optional): /envs/dev/**

Trigger: Automatic

Policy requirement: Optional

17. Click **Save**

18. Repeat these steps to create two more optional build validation policies:

Prod:

Build pipeline: pr-plan-prod

Path filter: /envs/prod/**

Trigger: Automatic

Policy requirement: Optional

Test:

Build Pipeline: pr-plan-test

Path filter: /envs/test/**

Trigger: Automatic

Policy requirement: Optional

Edit build policy

Enabled
☒ On

Build pipeline *
pr-plan-dev

Path filter (optional)
/envs/dev/**

Trigger
☒ Automatic (whenever the source branch is updated)
☐ Manual

Policy requirement
☐ Required
Build must succeed in order to complete pull requests.
☒ **Optional**
Build failure will not block completion of pull requests.

Build expiration
☐ Immediately when main is updated

Build Validation 3			
Validate code by pre-merging and building pull request changes.			
Enabled	Name ↑	Path filter	Trigger
☒ On	pr-plan-dev Optional	/envs/dev/**	Automatic Expires after 12 hours
☒ On	pr-plan-prod Optional	/envs/prod/**	Automatic Expires after 12 hours
☒ On	pr-plan-test Optional	/envs/test/**	Automatic Expires after 12 hours

Verify each new pipeline triggers correctly (three small PRs)

You will create three tiny PRs (one per env folder) to prove triggers work.

Keep the change harmless. The goal is trigger validation.

Note: In this lab, we observe automatic, but optional, PR plan runs. If you want to make these mandatory, they can later be added as branch policies — but that governance decision is intentionally out of scope here.

DEV trigger test

19. Create branch:

```
cd c:\ado-repo\env-roots
git checkout main
git pull
git branch -D pr-dev-plan
git checkout -b pr-dev-plan
```

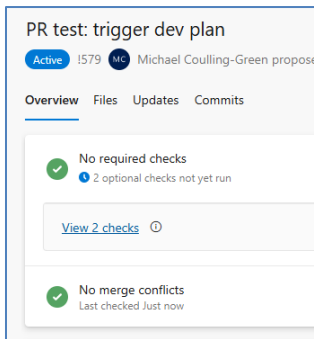
20. Make a tiny edit under envs/dev/ by adding a comment to envs/dev/main.tf

21. Commit + push:

```
git add envs/dev/main.tf
git commit -m "PR test: trigger dev plan"
git push -u origin pr-dev-plan
```

22. Create a PR to main

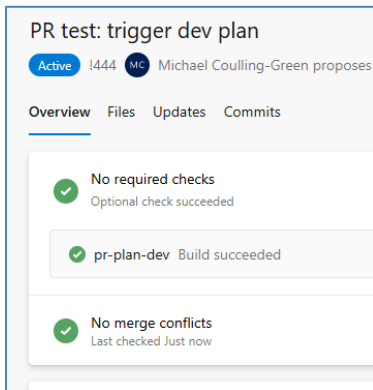
23. Switch to PR and select the triggered **pr-plan-dev**.



24. Click on 'View checks' and grant requested permissions as this is a first-run of this pipeline

25. Expected result: Only the **dev** PR plan pipeline runs for the PR.

26. Allow the no-op plan to complete



27. **Important:** Do not *complete* the PR. Completing the PR will merge the change, trigger the change-all pipeline and commit the changes to main, which is not the goal at this stage.

28. Abandon the PR (do not complete/merge).

29. Recovery from accidental merge (**Only if needed**)...

Allow the triggered no-op change-all pipeline to run to completion or cancel it. Then run ...

```
cd c:\ado-repo\env-roots
git checkout main
git pull
git revert -m 1 HEAD --no-edit
git push
```

Again, allow the triggered no-op change-all pipeline to run to completion or cancel it.

Skip the next step

30. Delete the branch..

```
cd c:\ado-repo\env-roots
git checkout main
git branch -D pr-dev-plan
git push origin --delete pr-dev-plan
```

TEST trigger test

31. Repeat the same steps 19-30 but referencing **test**.

PROD trigger test

32. Repeat the same steps 19-30 but referencing **prod**.

Phase 3 - Retire the change-all Pipeline

33. Disable the single change pipeline, **change-all** (.azure-pipelines/promote.yml) to avoid duplicate runs once environment-scoped change pipelines are introduced.

34. In Azure DevOps, go to Pipelines → Pipelines → All → change-all

35. Edit the pipeline → ... **edit**

36. Select ... (More actions) → settings.

37. Under Processing of new run requests, select Disabled.

38. Click **Save**.

Phase 4 - Review and release Environment-Scoped Pipelines

39. Instead of one pipeline that runs all environments, you will now use pre-provisioned environment-scoped pipelines.

40. Review the following pipelines in the **artifacts\pipelines\change** folder ...

- change-dev.yml
- change-test.yml

- change-prod.yml

41. Each pipeline:

- targets one environment only
- runs Terraform only for that environment
- is triggered only when that environment's configuration changes
- has a feature selection parameter, used to vary functionality, currently set at 1

42. Copy the supplied YAML from the lab artifacts into your local env-roots ...

From: **artifacts\pipelines\change**

change-dev.yml
change-prod.yml
change-test.yml

To: **env-roots\azure-pipelines**

43. At **env-roots**; Create Branch, Commit and push ...

```
cd c:\ado-repo\env-roots
git checkout main
git pull
git branch -D add-env-change-pipelines
git checkout -b add-env-change-pipelines
git add .azure-pipelines/change-*.yaml
git commit -m "Add env promotion pipelines"
git push -u origin add-env-change-pipelines
```

44. Complete the PR and commit to main.

45. In Azure DevOps > Pipelines, create and save without running, pipeline references for each YAML file.

change-dev.yml
change-prod.yml
change-test.yml

46. Rename each pipeline to:

change-dev
change-prod
change-test

Pipelines	
Recent	All Runs
All pipelines	
Name	
☒	change-all #20260126.7 • Revert "Merged PR 580: PR test: trigger test plan"
☐	change-dev No runs yet
☐	change-prod No runs yet
☐	change-test No runs yet
☒	pr-plan-dev #20260126.1 • PR test: trigger dev plan
☒	pr-plan-prod #20260126.1 • PR test: trigger prod plan
☒	pr-plan-test #20260126.1 • PR test: trigger test plan

Important: Double-check that you have selected the **change** yaml files, not **plan**.

Review the following status:

- Each environment change pipeline runs **only when its config changes**
- Dev can advance independently
- Test and prod require explicit change approval
- Approval gates are meaningful
- No environment can “jump ahead” *accidentally*, it must be a deliberate action

End-to-End Perspective (Read Before Moving On)

You have now seen all the components involved in a controlled change flow, even though they were introduced in stages. When viewed end-to-end, a typical change follows this sequence:

*The following describes the conceptual flow you have built, **not** a new set of steps to perform now.*

1. An environment change is made, the adoption of a new module version for example.
2. A pull request is opened proposing that environment change.
3. An **environment-scoped PR plan pipeline** runs, showing what would change if the proposal were accepted.
4. The plan is reviewed while the change is still only a proposal.
5. The pull request is merged - this is the moment the change is accepted.
6. An **environment-scoped change pipeline** runs for that environment.
7. An approval is requested where required (test and prod).
8. Terraform applies the change.

Each environment follows this same flow independently. Nothing is changed automatically, and nothing runs without a deliberate action somewhere in the chain.

A full walkthrough of this sequence — from module release through to applied promotion — will take place shortly.

Reflection: Something Still Is not Explicit

As at now, all environments are updated selectively and then pipelines run with approval requests where appropriate. Nothing happens accidentally.

And yet, something important is still missing. Pause for a moment and consider this question carefully:

Where did the decision to change this environment actually happen?

Not where it was *executed*. Not where it was *approved*. Not where Terraform *applied* changes. Where did someone explicitly say: “We are changing this environment now.”

You may find that the answer is unclear. The change may have been inferred from: a version change, a pipeline trigger, an approval click.

But inference is different from declaration. At small scale, this ambiguity is tolerable. At larger scale, it becomes dangerous. We still need to address one missing requirement:

How do we make change intent explicit — and ensure automation cannot act without it?

Do not try to solve this yet. Simply notice the gap.

Part 4. Forensic Review of a Production Promotion

Teaching Points

You will now promote a **new module version** to **production** using the promotion architecture you have already built.

Nothing in this lab is experimental.

Nothing in this lab is unsafe.

Nothing in this lab requires new tooling.

That is intentional.

The purpose of this run is not to *fix* the change process, but to **observe it carefully** and ask the hard question just posed ...

After a real production change, can we point to where the decision to change configuration was explicitly recorded?

Phase 1 – Create a New Module Version

In previous labs you designed and assembled environment change architecture, but you have not yet stepped through a complete production change from start to finish.

In this lab, you will deliberately exercise that complete architecture for the first time and observe how a real production change might unfold.

Release a new Security module version

In the **terraform-azure-module-security** module repository:

1. Open the existing **locals.tf**
2. Update the **manager value** from 'Michael Coulling-Green (SecMod v1.0.0)' to '**Peter Smith (SecMod v1.0.1)**'

This is a deliberately boring change. The change itself is not the point.

Commit and tag

From the **terraform-azure-module-security** module repository:

```
cd c:\module-repos\terraform-azure-module-security
git add locals.tf
git commit -m "Update manager"
git push origin main
git tag v1.0.1
git push origin v1.0.1
```

Confirm that the new tag exists remotely.

At this point, you have **published a new module version** — nothing more.

3. Take a moment to consider:
 - Have you stated *anywhere* that v1.0.1 is approved for production?
 - Or have you simply created a version that *could* be used?

Phase 2 – Promote new security module to PROD

Promotion in this architecture happens by **pinning a module version in an environment root**.

Create a branch in env-roots

4. In the env-roots repository:

```
cd c:\ado-repo\env-roots
git checkout main
git pull
git branch -D change-prod-1
git checkout -b change-prod-1
```

This branch exists solely to propose a production change.

Update the PROD module reference

5. Open: envs/prod/**main.tf**
6. Locate the **security** module source and update the version reference:

```
?ref=v1.0.0 > ?ref=v1.0.1
```

Do **not** change dev or test.

This is a **deliberate production change**, not an accidental leapfrog.

7. Ask yourself:

- Does changing ?ref= **declare** a promotion decision?
- Or does it **create** a configuration change that **results in** promotion later?

Both answers may feel partially true — note that tension.

Commit and push

```
git add envs/prod/main.tf
git commit -m "Promote Sec Mod v1.0.1 to PROD"
git push -u origin change-prod-1
```

Phase 3 – Pull Request and Validation

Create the Pull Request

8. In Azure DevOps:

- Create a PR targeting main
- Review the diff carefully by looking at the changed files
- Observe the PR validation pipeline running

9. Look at the PR and ask:

- If someone read this PR in six months' time, would they clearly see:

“This is a production promotion decision”?

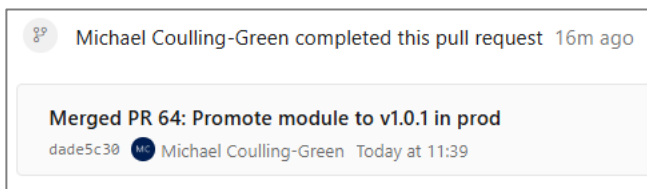
- Or would they merely infer that meaning from context?

Merge the PR

10. Complete the PR.

11. Record:

- PR number
- Author
- Merge date/time
- Secure Hash Algorithm (SHA)



Phase 4 – Observe the Production Pipeline

Observe pipeline execution

12. Navigate to: Pipelines → Pipelines

13. Open the **change-prod** run triggered by the **merge to main**.

14. Confirm:

- Only the production pipeline runs
- The prod environment is targeted
- The correct module version is applied

Mechanically, everything should look correct.

15. Ask yourself:

- Where does Azure DevOps explicitly state that this run represents a **promotion decision**?
- Or are you inferring that from file paths, YAML names, and context?

Phase 5 – Approval and Execution

Approve the production gate

16. Grant the permissions needed as this is the first run of this pipeline.

17. When the pipeline pauses:

- Review the approval request
- Approve (or observe approval)

18. Record:

- Who approved
- When

19. Consider carefully:

- Does this approval say:
“We choose to promote production to vX.Y.Z”?
- Or does it say:
“This pipeline is allowed to continue”?
That distinction matters.

Observe Terraform apply

20. Allow the pipeline to complete.

Confirm:

- Apply succeeds
- The manager value is updated in production

Everything has worked exactly as designed.

Phase 6 – Forensic Review

Gather your evidence

21. Collect:

- Module version referenced in production configuration

Where to find it:

In the production environment configuration:

- envs/prod/main.tf
- The module source reference (for example ref = "v1.0.1")

This change will also be visible in the production pull request diff.

Why it matters:

This shows what version of the module production is configured to use.

It represents the *technical content* of the promotion — but not the decision itself.

On its own, this is just configuration. It does not tell you when or why production adopted this version.

- Production pull request

Where to find it:

In Azure DevOps...

- Repos → Pull Requests → Completed
- Locate the PR that modified envs/prod/main.tf

Why it matters:

The production pull request represents the formal proposal to change production configuration.

It shows:

- who proposed the change
- what was changed
- when the change was accepted

However, the PR does not confirm that production was actually updated — only that the change was approved to be merged.

- Promotion pipeline run

Where to find it:

In Azure DevOps...

- Pipelines → Runs
- Locate the run of the production promotion pipeline triggered by the PR merge

The run will typically reference the commit SHA from the merged PR.

Why it matters:

This pipeline run represents execution.

It is the point where automation acted on the accepted change.

This answers the question:

“When did the platform attempt to promote production?”

But it still does not tell you why that promotion was authorised.

- Approval record

Where to find it:

In Azure DevOps ...

- Environments → prod → History
- Select the relevant pipeline run and view its approvals

Why it matters:

The approval record shows who allowed execution to proceed and when.

This is the strongest human signal in the system — but it is still procedural, not declarative. An approval allows execution; it does not state *intent* in a durable, auditable form.

- Commit SHA

Where to find it:

- In the merged production pull request, or
- In the promotion pipeline run details

It will look like a short hexadecimal string (for example `dade5c30`).

Why it matters:

The commit SHA is the immutable identifier for this promotion event.

It cryptographically ties together:

- the production PR
- the pipeline run
- the approval
- the resulting state change

It proves *what happened*, but not *why it happened*.

You now have a full promotion trail.

Look for explicit intent

22. Try to find **one artifact** that explicitly states:

“We decided to promote production to vX.Y.Z.”

Do not infer. Do not combine clues. Look for a single, clear declaration.

Expected Outcome

Your conclusion should be:

The promotion was deliberate, controlled, approved, and successful.

However, the decision to promote exists only as inferred behaviour, not as an explicit artifact.

Why This Matters

This lab proves something subtle but important:

- **Control** is not the same as **clarity**
- **Deliberate action** is not the same as **declared intent**
- **Successful automation** can still fail an audit question

Transition to next lab

In the next lab, you will redesign this flow so that:

- Changes are categorized
- Change intent is declared **before** execution
- Pipelines refuse to run without it
- Promotion decisions are explicit, durable, and auditable

For now, resist the urge to fix anything. You have just discovered *why* a fix is needed.

© QA 2026

Copyright

© 2026 QA Michael Coulling-Green. All rights reserved. These lab materials are provided as part of an authorised training course. Course attendees may reuse these materials for their own personal learning and reference outside of the training session. Unauthorised copying, redistribution, publication, or use of these materials to deliver training is prohibited without prior written permission from the author.

© QA 2026